

SSD 물리적 단편화의 정량적 제어 기법

이승윤⁰¹, 김상훈¹

¹ 아주대학교 인공지능학과

soccer1356@ajou.ac.kr, sanghoonkim@ajou.ac.kr

A Quantitative Control Mechanism for Physical Fragmentation in SSDs

Seungyun Lee⁰¹, Sang-Hoon Kim¹

¹Department of Artificial Intelligence, Ajou University

soccer1356@ajou.ac.kr, sanghoonkim@ajou.ac.kr

요 약

현대 고성능 SSD 는 다수의 채널과 다이를 활용한 내부 병렬성으로 최고 성능을 달성하지만, I/O 환경에서 발생 하는 물리적 단편화는 이러한 병렬성을 훼손하여 치명적인 성능 저하를 유발한다. 기존 연구들은 상용 SSD FTL 의 블랙박스 특성으로 인해 물리적 단편화를 간접적으로 추정하는 데 그쳤다. 이를 극복하기 위해 본 논문은 소프트웨어 정의 NVMe 에뮬레이터인 NVMeVirt 의 FTL 내부를 직접 제어하여, 물리적 단편화를 정량적으로 유도하고 분석할 수 있는 SPF(Stochastic Physical Fragmentation) 모델을 제안한다.

1. 서 론¹

현대 SSD는 다수의 채널과 다이로 구성된 복잡한 내부 구조를 통해 높은 I/O 병렬성을 제공한다. 이상적인 환경에서는 FTL이 라운드로빈 방식으로 데이터를 각 다이에 균등하게 분산 배치하여 최대 병렬성을 달성한다. 그러나 실제 워크로드에서는 물리적 단편화로 인해 순차 읽기 성능이 크게 저하된다[1]. 물리적 단편화 연구는 오랫동안 상용 SSD FTL의 블랙박스 특성으로 인해 제약을 받아 왔다. 내부 데이터 배치 상태를 직접 관측할 수 없기 때문에, 물리적 단편화를 원하는 수준으로 유도하거나 그 영향을 정량적으로 분석하는 것이 사실상 불가능하였다[3]. 기존의 SSD 에이징 도구들은 파일시스템 수준의 논리적 단편화는 재현할 수 있으나, 물리적 단편화를 독립적으로 제어하는

기능은 제공하지 않는다[4,5]. 본 논문은 이러한 한계를 극복하기 위해 소프트웨어 정의 NVMe 에뮬레이터인 NVMeVirt[6]를 기반으로 물리적 단편화를 정량적으로 제어하는 SPF(Stochastic Physical Fragmentation) 모델을 제안한다. 본 연구의 주요 기여는 다음과 같다: (1) FTL 내부를 직접 제어하여 물리적 단편화를 0%에서 100%까지 정밀하게 제어하는 메커니즘 제안, (2) 세 파라미터의 독립적 영향 분석, (3) 물리적 단편화와 순차 읽기 성능 저하의 관계를 입증한다.

2. 배경 및 관련 연구

2.1 물리적 단편화

물리적 단편화는 논리적으로 연속된 데이터가 이상적인 라운드로빈 배치에서 벗어나 특정 다이에 불균등하게 집중되는 현상을 의미한다[1]. 물리적 단편화가 존재하면 순차 읽기 명령이 복수의 페이지를 병렬로 읽지 못하고 직렬로 처리해야 하므로, 장치의 내부 병렬성이 감소하고 처리량이 저하된다.

2.2 관련 연구

* 이 성과는 과학기술정보통신부의 재원으로 한국연구재단 (RS-2025-16067415)과 과학기술정보통신부 및 정보통신기획평가원의 인공지능융합혁신인재양성사업 (IITP-2026-RS-2023-00255968)의 지원을 받아 수행된 연구임

Jun et al.[1]은 플래시 SSD에서 파일 단편화의 성능 영향이 논리적 단편화보다 물리적 단편화에서 비롯됨을 실증하였다. 그러나 이 연구는 상용 SSD를 사용하였기 때문에 물리적 배치 상태를 직접 측정하지 못하고 간접적인 방법으로 영향을 추정하였다. Geriatrix[3]는 현실적인 파일시스템 에이징 환경을 제공하는 도구로, 실제 워크로드 분포를 기반으로 파일 생성·삭제를 반복하여 논리적 단편화를 재현하지만, 직접적으로 물리적 단편화 제어하는 기능은 없다.

3. SPF 모델 설계

물리적 단편화를 유발하는 간단한 방법은 장치 전체에 걸쳐 데이터를 무작위로 재배포하는 것이다. 그러나 이러한 접근 방식은 단편화의 정도를 정밀하게 제어하기 어렵게 만들며, 중간 수준의 단편화와 심각한 수준의 단편화를 구별하기 어렵게 만든다.

이러한 한계를 극복하기 위해, 우리는 세 가지 직관적인 파라미터를 기반으로 한 새로운 모델을 설계하였다.

3.1 통찰 1: 충돌 확률 Pdegree

우리가 목표로 한 것은 배치가 이상적인 라운드로빈 배치에서 얼마나 자주 벗어나는지를 결정할 수 있는 파라미터였다. 이러한 요구사항은 Pdegree의 도입 동기가 되었다: 각 쓰기마다 모델은 $[0, 100]$ 범위의 난수를 생성한다. 만약 $R \leq Pdegree$ 이면, 쓰기는 다음 다이가 아닌 이전 쓰기와 동일한 다이에 배치된다 ($P_{degree} = 0$ 인 경우는 제외). 결과적으로, Pdegree = 0일 때 모델은 엄격한 라운드로빈 배치를 유지하며, 반대로 Pdegree = 100일 때는 모든 쓰기를 단일 다이로 집중시킨다.

3.2 핵심 통찰 2: 버스트 길이 Lburst

물리적 단편화의 빈도에 더해, 개별 이벤트의 구조적 영향을 모델링하기 위해 도입한 Lburst는 충돌 발생 시 동일한 다이에 할당되는 연속적인 페이지 수를 정의한다. Lburst를 작은 값으로 설정하면 Pdegree가 단편화 이벤트를 트리거하더라도 그 영향은 미미하게 유지된다. 결과적으로 Pdegree가 증가함에 따라 성능은 비교적 점진적으로 저해된다. 그러나 더 큰 Lburst 값은 단 한 번의 충돌 이벤트로도 여러 페이지를 특정 다이에 집중시키게 만든다. 이는 대상 다이에 심각한 다이 충돌 부하를 가하여 급격한 성능 하락을 초래한다.

3.3 핵심 통찰 3: 스트림 수 Nstreams

실제 환경에서 SSD는 동시 I/O 요청으로 인해 발생하는 인터리빙된 데이터 스트림을 빈번하게 처리한다. 일반적인 FTL은 도착 순서에 따라 라운드로빈 방식으로 다이를 할당하기 때문에, 여러 프로세스의 동시 쓰기는 물리적 단편화로 이어질 수 있다. 예를 들어, 파일 A의 연속된 페이지 사이에 파일 B의 쓰기가 끼어들면, FTL의 라운드로빈

순서가 흐트러져 파일 A의 인접한 두 페이지가 동일한 다이에 배치될 수 있다. 이 경우 순차 읽기 시 직렬로 처리해야 하므로 성능이 저해된다. 이를 모사하기 위해 스트림 수 파라미터 Nstreams를 도입하였다. 논리 주소 공간을 Nstreams개의 동일 크기 세그먼트로 분할하고, 라운드로빈 방식으로 각 세그먼트에서 번갈아 데이터를 기록한다. Nstreams = 1이면 단일 스트림으로 인터리빙이 없으며, Nstreams가 증가할수록 서로 다른 스트림의 데이터가 FTL 수준에서 점점 더 깊이 뒤섞인다.

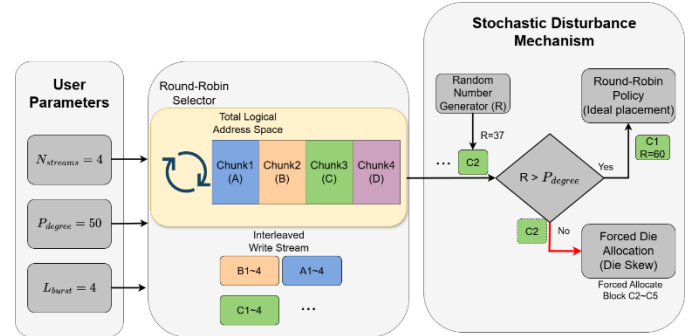


그림 1. SPF 모델 Overview

3.5 SPF 모델 통합

SPF(Stochastic Physical Fragmentation) 모델은 Pdegree, Lburst, Nstreams 세 파라미터로 구성된다. Pdegree는 충돌이 얼마나 자주 일어나는지를, Lburst는 충돌 한 번이 얼마나 큰 구조적 부하를 남기는지를, Nstreams는 스트림 간 간섭이 얼마나 깊은지를 각각 독립적으로 결정한다. 예를 들어 Nstreams = 4, Pdegree = 50, Lburst = 4로 설정하면, 4개의 파일 스트림이 FTL에서 인터리빙되면서 쓰기 절반은 라운드로빈을, 나머지 절반은 동일 다이에 4페이지씩 연속 배치하는 패턴이 만들어진다.

4. 평가

4.1 실험 환경

본 실험은 Linux 커널 5.10.0 위에 NVMeVirt를 기반으로 SPF 모델을 구현하였다. 실험 서버는 Intel Xeon Gold 5215 CPU(2.50GHz, 10코어) 2소켓과 379GB DRAM으로 구성되었다. NVMeVirt 인스턴스는 16개의 다이(8채널 × 2웨이), 32KiB 페이지 크기를 갖는 128GiB 용량의 SSD를 에뮬레이션하도록 설정하였다. 성능 측정에는 fio 벤치마킹 도구를 사용하였으며, OS 캐시의 영향을 제거하기 위해 O_DIRECT 플래그와 페이지 캐시 플러시를 적용하였고, 커널 수준의 readahead을 비활성화하였다. 또한 호스트 측 I/O 버퍼 단편화를 방지하기 위해 hugepage 기반의 연속 버퍼를 할당하였으며, iodepth는 32로 설정하였다.

4.2 파라미터 개별 영향 분석

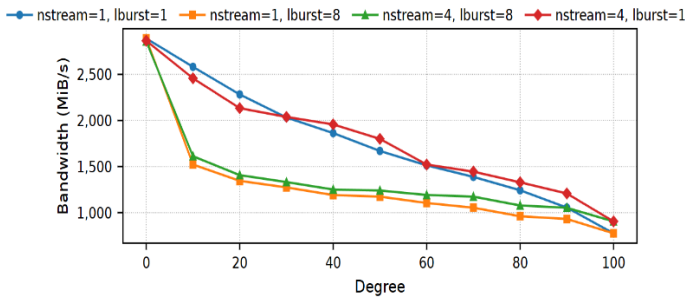


그림 2. Degree 0~100 변화에 따른 처리량

그림 2에서 볼 수 있듯이, Pdegree가 증가함에 따라 대역폭은 지속적으로 감소한다. 성능 하락의 규모는 Lburst 및 Nstreams와 같은 다른 파라미터의 설정에 따라 달라지지만, Pdegree가 증가함에 따른 대역폭의 전반적인 하락 추세는 모든 조건에서 일관되게 유지된다. 이러한 동작은 Pdegree가 낮은 범위에서는 모델이 여러 다이에 걸쳐 데이터를 비교적 균등하게 분배하여 SSD의 내부 병렬성을 보존하기 때문에 발생한다. 반대로 Pdegree를 증가시키면 특정 다이에 대한 I/O 요청 집중도가 심화된다. 특히 Pdegree가 높은 범위에서는 이러한 집중이 내부 병렬성을 심각하게 악화시켜, 대역폭의 상당한 하락과 지연 시간의 급증으로 이어진다. 결과적으로 Pdegree는 물리적 단편화로 인한 성능 저하 추세를 결정하는 가장 지배적인 변수 역할을 한다.

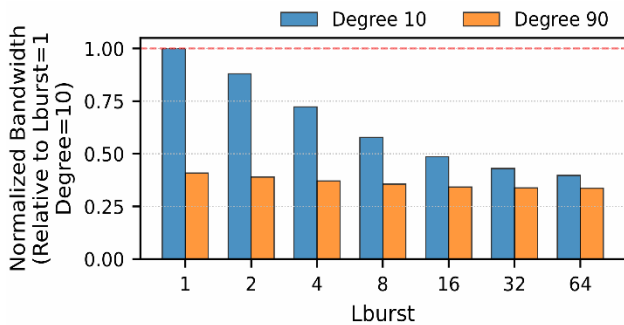


그림 3. Lburst 변화에 따른 처리량

그림 3에 나타난 조건에서, Lburst가 1에서 64로 증가함에 따라 대역폭은 감소한다. 이러한 감소는 Lburst 값이 커질수록 다이 충돌의 지속 시간이 연장되어 병목 현상을 악화시킨다는 사실에서 기인한다. 대조적으로, 이미 심각한 단편화가 진행된 조건(Pdegree = 90)에서는 Lburst 증가로 인한 추가적인 대역폭 하락이 상대적으로 더 작다. 이 현상은 대부분의 데이터가 이미 특정 다이에 쏠려 있어, 극도로 긴 다이 충돌이 기준으로 자리 잡았기 때문에 발생한다. 그럼에도 불구하고 우리는 대역폭의 전반적인 하락 추세를 일관되게 관찰할 수 있으며, 이는 Lburst가

기존 다이 충돌의 구조적 심각도를 증가시키는 데 독립적인 역할을 수행함을 확인해 준다.

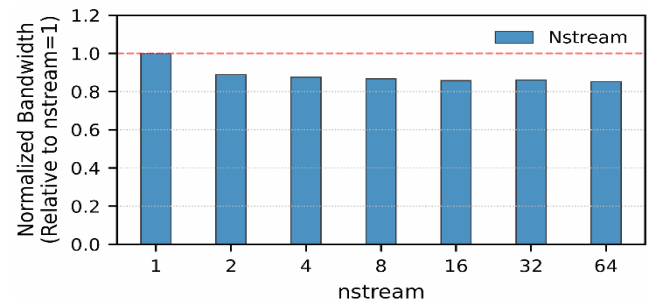


그림 4. Nstreams 변화에 따른 처리량

Nstreams는 여러 데이터 스트림이 동시에 기록되는 상황을 시뮬레이션하는 파라미터이다. 그림 4에서 볼 수 있듯이, 충돌 빈도와 길이를 고정한 상태에서 Nstreams가 증가하면서 서로 다른 스트림의 데이터가 점점 더 뒤섞이게 된다. 이는 FTL 수준의 인터리빙을 심화시키고 물리적 배치를 분산시켜 내부 병렬성을 악화시킨다. 그러나 이러한 성능 저하의 규모는 Pdegree나 Lburst에 비해 상대적으로 완만하다. 결과적으로 Nstreams는 극단적인 병목 현상을 직접적으로 유발하는 변수라기보다는, 동시 워크로드에서의 상호 간섭을 현실적으로 재현하기 위한 보조적인 파라미터로 보는 것이 타당하다.

5. 결론

본 논문은 SSD의 물리적 단편화를 정량적으로 제어할 수 있는 SPF 모델을 제안하였다. 실험 결과, 단편화 수준이 높아질수록 내부 병렬성이 악화되고 순차 읽기 성능이 유의미하게 감소함을 확인하였다. 특히 SPF는 기존에 직접 관측하기 어려웠던 물리적 단편화를 실험적으로 재현할 수 있다는 점에서 의미가 크다. 이 연구는 SSD 벤치마크 설계, 저장장치 성능 평가, 그리고 향후 단편화 완화 기법 개발에 활용될 수 있다.

참고문헌

- [1] Y. Jun, S. Park, J. Kang, S. Kim, and E. Seo, "We Ain't Afraid of No File Fragmentation: Causes and Prevention of Its Performance Impact on Modern Flash SSDs," *USENIX FAST*, pp. 193–208, 2024.
- [2] G. Haas, B. Lee, P. Bonnet, and V. Leis, "SSD-IQ: Uncovering the Hidden Side of SSD Performance," *VLDB*, 2025.
- [3] S. Kadekodi, V. Nagarajan, and G. R. Ganger, "Geriatric: Aging What You See and What You Don't See," *USENIX ATC*, pp. 691–704, 2018.
- [4] S.-H. Kim et al., "NVMeVirt: A Versatile Software Defined Virtual NVMe Device," *USENIX FAST*, 2023.